Winter 2026

# STAT 390 - Presentation 4

**Team 3**

Leah Tan, Xiaolin Liu, Elsa Steen Koppell, Leslie Gonzalez-Flores, and Sofie Langenhuizen

GitHub Repo

# Roles & Responsibilities

**Our Team**

**Leah**
**Consultant**

*Responsible for suggesting changes and room for improvement in methodology, execution*

**Sofie**
**Coder**

*Expert in the code, produces and executes the model, making necessary edits to any redundancies*

**Xiaolin**
**Coder**

*Expert in the code, produces and executes the model, making necessary edits to any redundancies*

**Leslie**
**Mathematician**

*Expert in the math behind the model, suggest edits to methodology and provide justification for parameters*

**Elsa**
**Mathematician**

*Expert in the math behind the model, suggest edits to methodology and provide justification for parameters*

Leah

Agenda

- Objective
- Hyperparameter Tuning
- Similarity of Models
- Small Data Memorization
- Takeaways
- Project Reflection

# Objective

Finalize machine learning model **to more accurately classify** the categories of **C-MIL** (Conjunctival melanocytic intraepithelial lesions)

- ➔ Presentation 3 resulted in major improvements, with an average of **91%** accuracy across all 5 splits

<u>Instated Changes:</u>

- ➔ Low-level embeddings of patches vs. resizing 224×224
- ➔ Fix data leakage among splits
- ➔ Gated Attention & Entropy Analysis

# Hyperparameter Tuning

- Optuna for Bayesian tuning
  - **64 trials**
  - 16 startup trials using **sobol sampling algorithm**
  - 30 epochs w/ early stopping
  - Pruning
    - MedianPruner
    - 6 warmup epochs
  - Takes in 5 different splits, each epoch calculates validation loss on each individual split
  - **Objective: minimize average validation loss**
  - Wide search space:
    - Initial Learning Rate: (3e-5, 1e-3)
    - Weight Decay: (1e-6, 2e-3)
    - Patch Projector Dropout: (0.15, 0.55)
    - Classifier Dropout: (0.15, 0.55)
    - Benign Class Weight: (2.0, 4.0)
    - Entropy Lambda: (1e-5, 1e-2)

**Optuna script:**
/projects/e32998/STAT390_Winter2026/Presentation4/Team3/base_code/optuna_training.py
**Sbatch file:**
/projects/e32998/STAT390_Winter2026/Presentation4/Team3/base_code/sbatch_files/run_optuna.sbatch

# Hyperparameter Tuning

- Changes made to optuna_tuning.py
  - Adding in early stopping based on hyperparameters specified in config file

```python
class EarlyStopper:
    def __init__(self, patience: int, min_delta: float, min_epochs: int):
        self.patience = patience
        self.min_delta = min_delta
        self.min_epochs = min_epochs
        self.best_loss = float("inf")
        self.epochs_without_improvement = 0

    def step(self, epoch: int, mean_val_loss: float) -> bool:
        """Returns True if training should stop."""
        if epoch < self.min_epochs:
            return False
        if mean_val_loss < self.best_loss - self.min_delta:
            self.best_loss = mean_val_loss
            self.epochs_without_improvement = 0
        else:
            self.epochs_without_improvement += 1
        return self.epochs_without_improvement >= self.patience
```

```python
def objective(trial: optuna.Trial) -> float:
    try:
        for epoch in range(max_epochs):
            if mean_val_loss < best_avg_val_loss:
                best_avg_val_loss = mean_val_loss
                best_epoch = epoch + 1
                best_epoch_fold_val_losses = list(epoch_val_losses)
                best_epoch_fold_val_accs = list(epoch_val_accs)

            trial.report(mean_val_loss, step=epoch)
            if trial.should_prune():
                trial.set_user_attr("pruned_epoch", epoch + 1)
                raise optuna.TrialPruned()

            # early stopping check (after pruning check)
            if early_stopper is not None and early_stopper.step(epoch + 1, mean_val_loss):
                print(f"  --> Early stopping triggered at epoch {epoch + 1} "
                      f"(no improvement for {early_stopper.patience} epochs)")
                trial.set_user_attr("early_stopped_epoch", epoch + 1)
                break
```

**Modified scripts:**
/projects/e32998/STAT390_Winter2026/Presentation4/Team3/base_code/optuna_training.py
/projects/e32998/STAT390_Winter2026/Presentation4/Team3/base_code/models.py
/projects/e32998/STAT390_Winter2026/Presentation4/Team3/base_code/config.py

# Hyperparameter Tuning

- **Runtime: 2d 13hr**
- **Pruned trials: 29**
- **Complete trials: 35**
  - 100% of complete trials stopped early
  - Max number of epochs: 29
- **Best trial: 5**
  - Best epoch: 9
  - Early stopped epoch: 18
  - Best average validation loss: 0.2215
    - Split 1: 0.1826
    - Split 2: 0.1276
    - Split 3: 0.1917
    - Split 4: 0.2930
    - Split 5: 0.1942

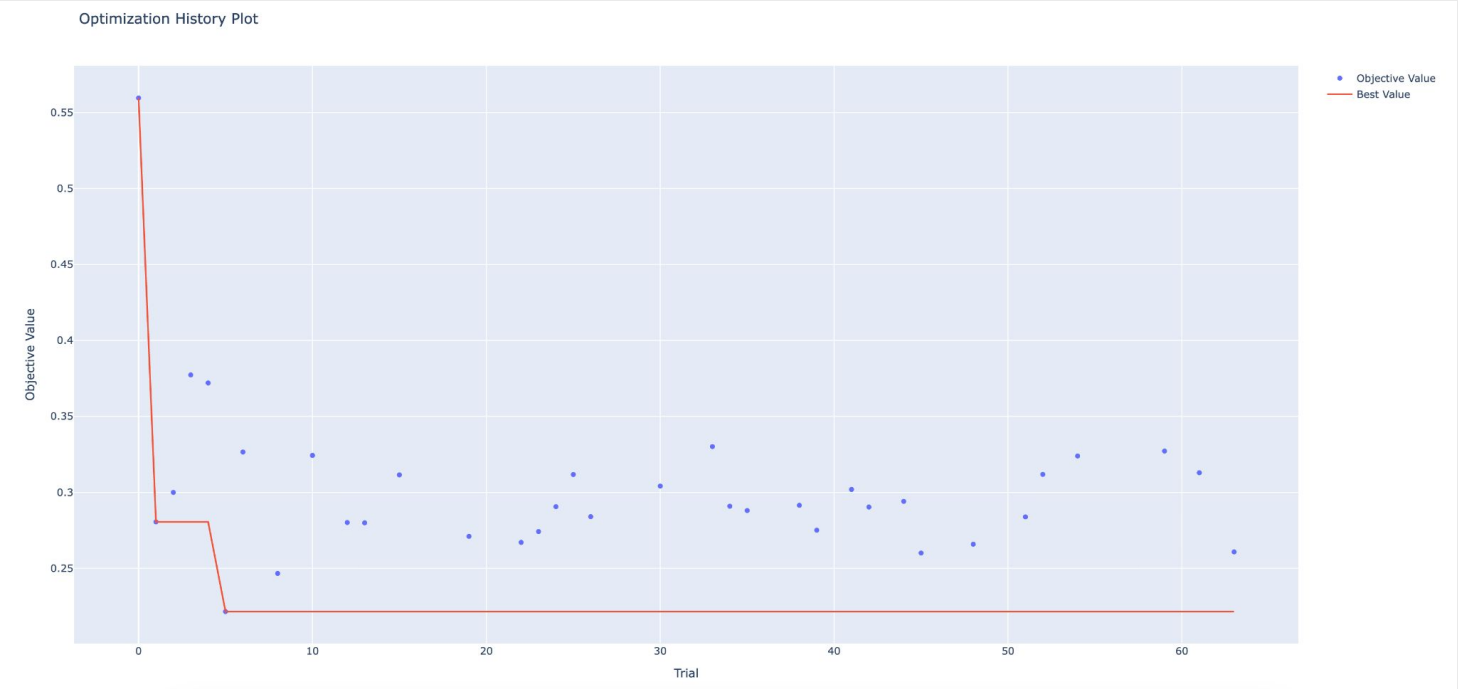
**Optuna study summary:**
/projects/e32998/STAT390_Winter2026/Presentation4/Team3/base_code/optuna_results_team3_20260311_214616/study_summary.json
**Optuna study output:**
/projects/e32998/STAT390_Winter2026/Presentation4/Team3/base_code/mar11_tuning_try3.out

# Hyperparameter Tuning

| Hyperparameter | Base Model | Optuna Study |
|---|---|---|
| Initial Learning Rate | 2.08e-4 | 5.225e-05 |
| Weight Decay | 3.05e-4 | 4.696e-04 |
| Patch Projection Dropout | 0.2599 | 0.511010 |
| Classifier Dropout | 0.2599 | 0.198204 |
| Benign Class Weight | 2.5285 | 2.136586 |
| Entropy Lambda | 0.001 | 9.976e-04 |

**Base model hyperparameters:**
/projects/e32998/STAT390_Krish/Code/Code9_no_leakage/config.py
**Optuna study best hyperparameters:**
/projects/e32998/STAT390_Winter2026/Presentation4/Team3/base_code/optuna_results_team3_20260311_214616/study_summary.json

# Hyperparameter Tuning




**Optimization history plot:**
/projects/e32998/STAT390_Winter2026/Presentation4/Team3/base_code/optuna_results_team3_20260311_214616/optimization_history.html

# Hyperparameter Tuning

Overall average accuracy of **87.2%**, average loss of **0.5334**

➔ Worse off than baseline average accuracy

| Data Split | Base Model | | Optimized Model | |
|---|---|---|---|---|
| | Test Loss | Test Acc | Test Loss | Test Acc |
| Split 1 | 0.1970 | 0.9444 | 0.1737 | 0.9444 |
| Split 2 | 0.5799 | 0.7895 | 1.2223 | 0.7368 |
| Split 3 | 0.2701 | 0.9474 | 0.8550 | 0.7895 |
| Split 4 | 0.0388 | 1.0000 | 0.0873 | 1.000 |
| Split 5 | 0.3368 | 0.8889 | 0.3287 | 0.8889 |

**Base model outputs:**
/projects/e32998/STAT390_Krish/Code/Code9_no_leakage/sbatch_files/test/_split2/_split3/_split4/_split5.out
**Optimized model outputs:**
/projects/e32998/STAT390_Winter2026/Presentation4/STAT390_Winter2026_Team3/base_code/mar14_test_/split1_retry/split2/split3/split4/split5.out

# Hyperparameter Tuning

**Changes made to config.py and models.py:**

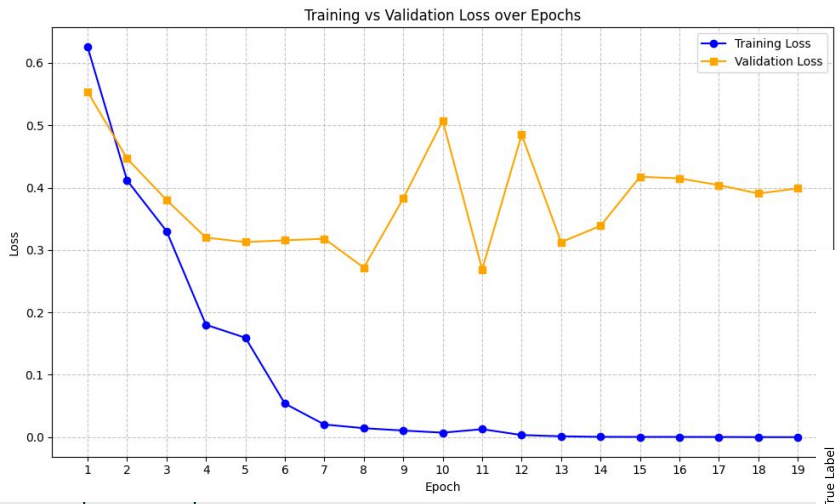
- ➔ Base script has a single *dropout* hyperparameter in configs
- ➔ Optuna tuning tunes two dropout hyperparameters: *patch_proj_dropout* and *classifier_dropout*
- ➔ Updated code in config and models to use both dropout parameters:

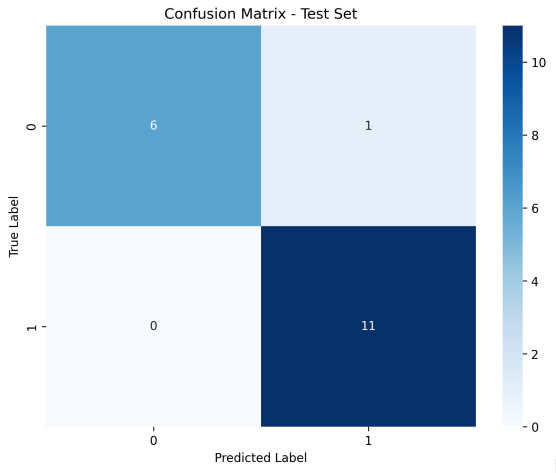
```python
def create_model(
    num_classes: int = None,
    embed_dim: int = None,
    dropout: float = None,
    pooled_dim: int = 4096,
) -> HierarchicalAttnMIL:

    if num_classes is None:
        num_classes = MODEL_CONFIG["num_classes"]

    if embed_dim is None:
        embed_dim = MODEL_CONFIG["embed_dim"]

    if dropout is None:
        from config import TRAINING_CONFIG
        dropout = TRAINING_CONFIG.get("dropout", 0.3)

    return HierarchicalAttnMIL(
        num_classes=num_classes,
        embed_dim=embed_dim,
        dropout=dropout,
        pooled_dim=pooled_dim,
    )
```

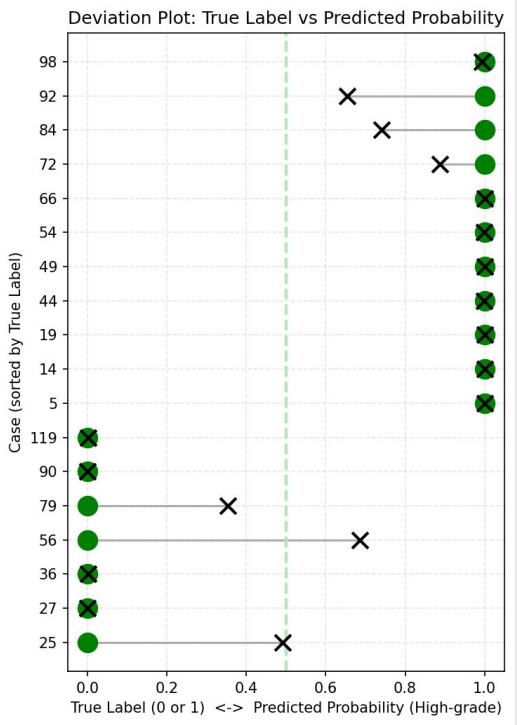
```python
def create_model(
    num_classes: int = None,
    embed_dim: int = None,
    patch_proj_dropout: float = None,
    classifier_dropout: float = None,
    pooled_dim: int = 4096,
) -> HierarchicalAttnMIL:

    if num_classes is None:
        num_classes = MODEL_CONFIG["num_classes"]

    if embed_dim is None:
        embed_dim = MODEL_CONFIG["embed_dim"]

    if patch_proj_dropout is None or classifier_dropout is None:
        from config import TRAINING_CONFIG
        patch_proj_dropout = TRAINING_CONFIG.get("patch_proj_dropout", 0.3)
        classifier_dropout = TRAINING_CONFIG.get("classifier_dropout", 0.3)

    return HierarchicalAttnMIL(
        num_classes=num_classes,
        embed_dim=embed_dim,
        patch_proj_dropout=patch_proj_dropout,
        classifier_dropout=classifier_dropout,
        pooled_dim=pooled_dim,
    )
```

**Optuna study best hyperparameters:**
/projects/e32998/STAT390_Winter2026/Presentation4/Team3/base_code/optuna_results_team3_20260311_214616/study_summary.json

# Optimal Model: Split 1

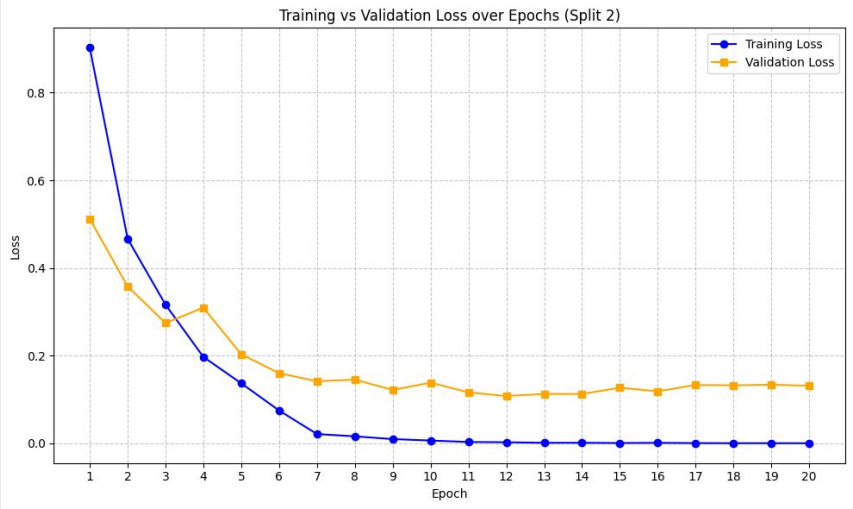


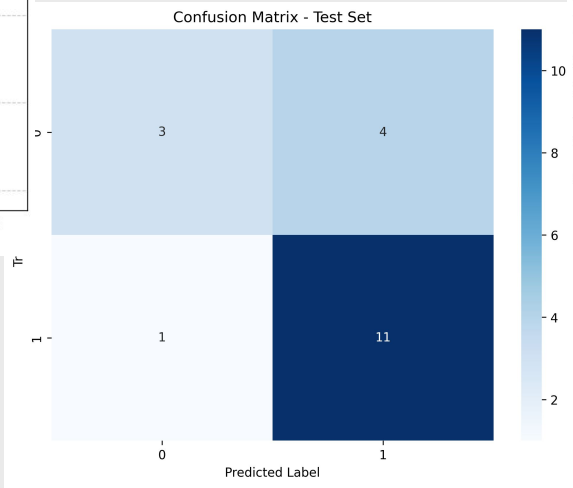



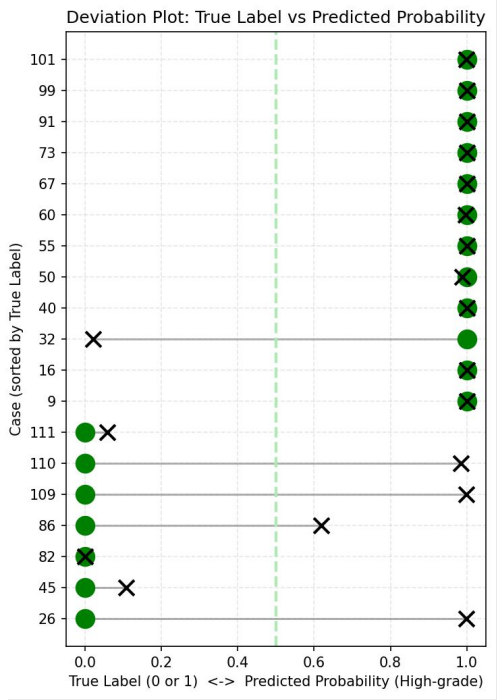




**Path to results:**
/projects/e32998/STAT390_Winter2026/Presentation4/STAT390_Winter2026_Team3/base_code/mar14_test_split1_retry.out

# Optimal Model: Split 2

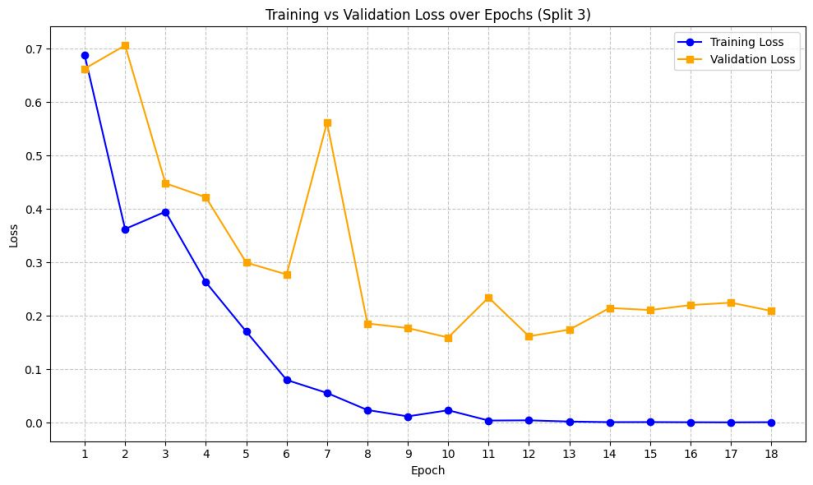


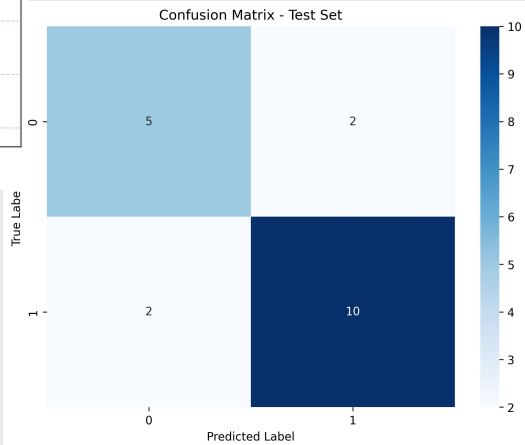



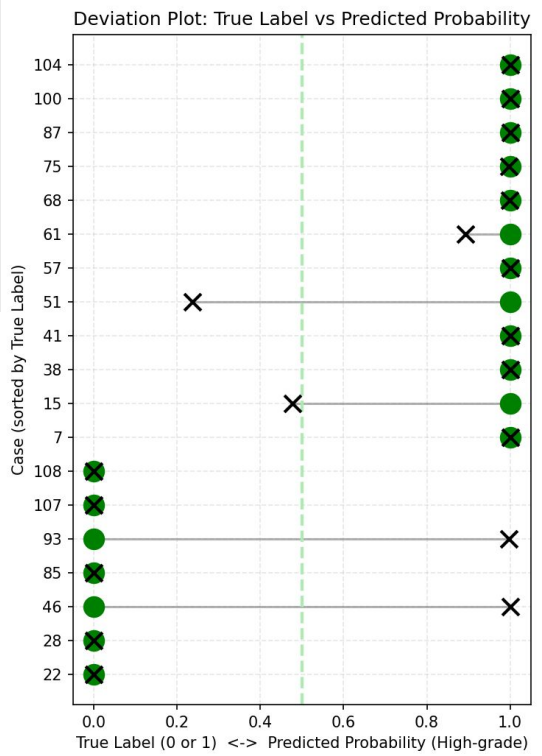




**Path to results:**
/projects/e32998/STAT390_Winter2026/Presentation4/STAT390_Winter2026_Team3/base_code/mar14_test_split2.out

# Optimal Model: Split 3

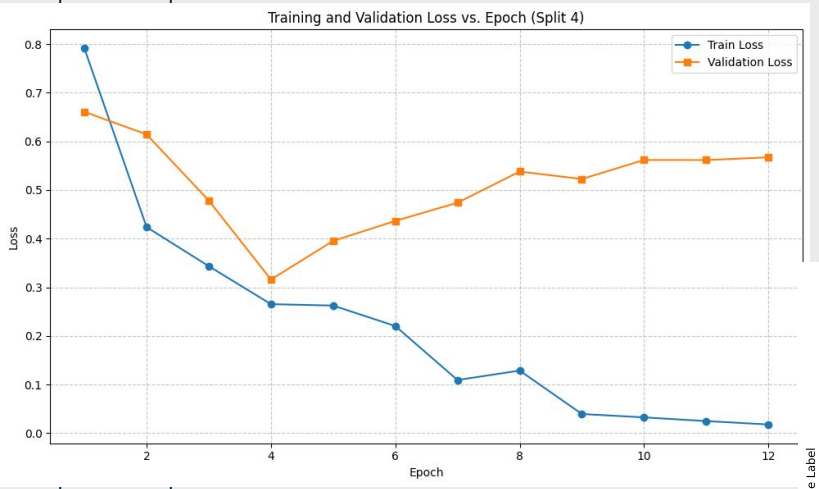


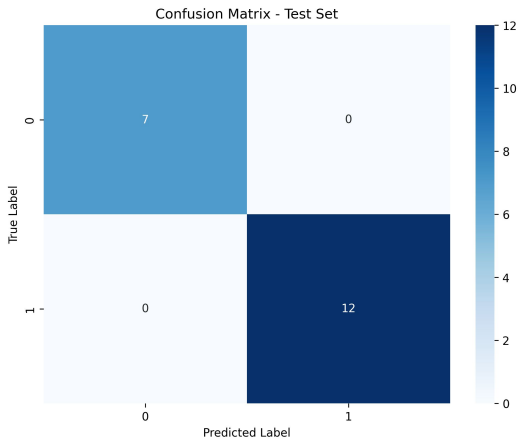



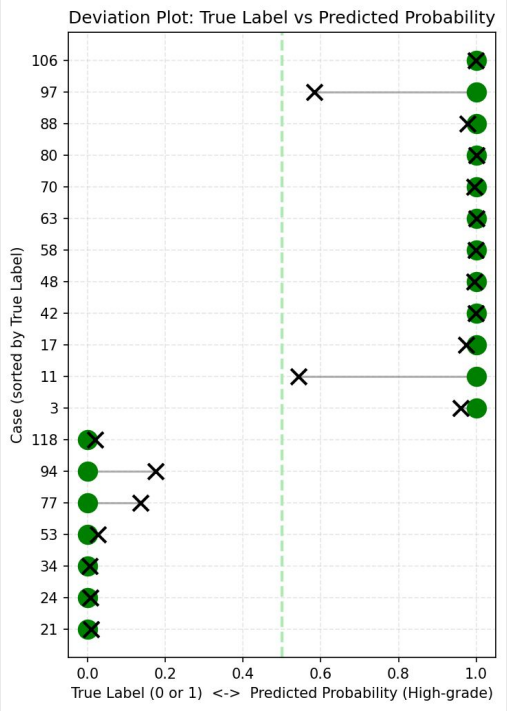




**Path to results:**
/projects/e32998/STAT390_Winter2026/Presentation4/STAT390_Winter2026_Team3/base_code/mar14_test_split3.out

# Optimal Model: Split 4

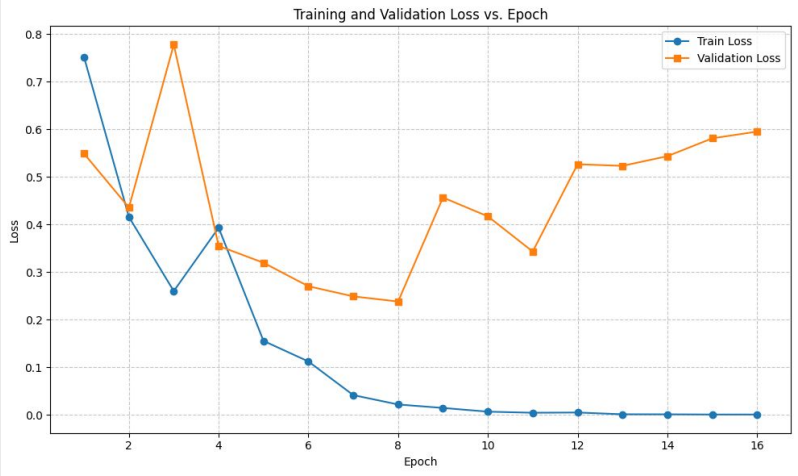


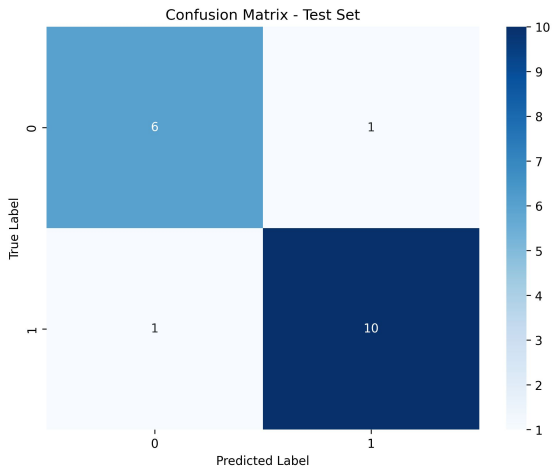



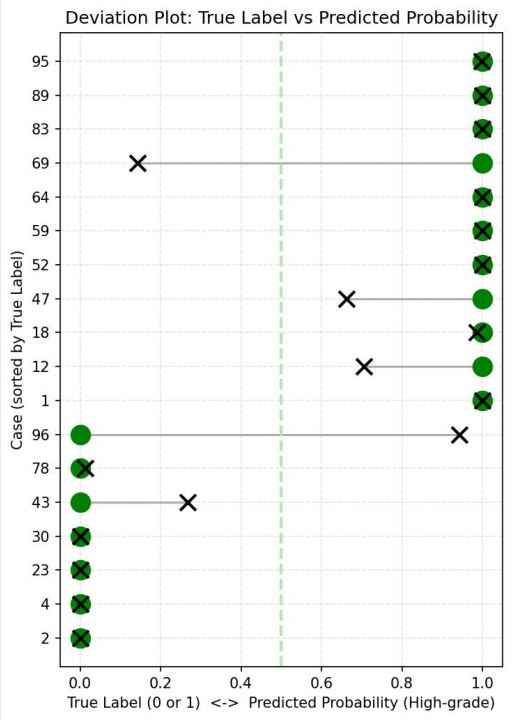




**Path to results:**
/projects/e32998/STAT390_Winter2026/Presentation4/STAT390_Winter2026_Team3/base_code/mar14_test_split4.out

# Optimal Model: Split 5




**Path to results:**
/projects/e32998/STAT390_Winter2026/Presentation4/STAT390_Winter2026_Team3/base_code/mar14_test_split5.out

# Optuna Model: Patch Attention Analysis

**Split 4 best** performing, **Split 2 worst** performing

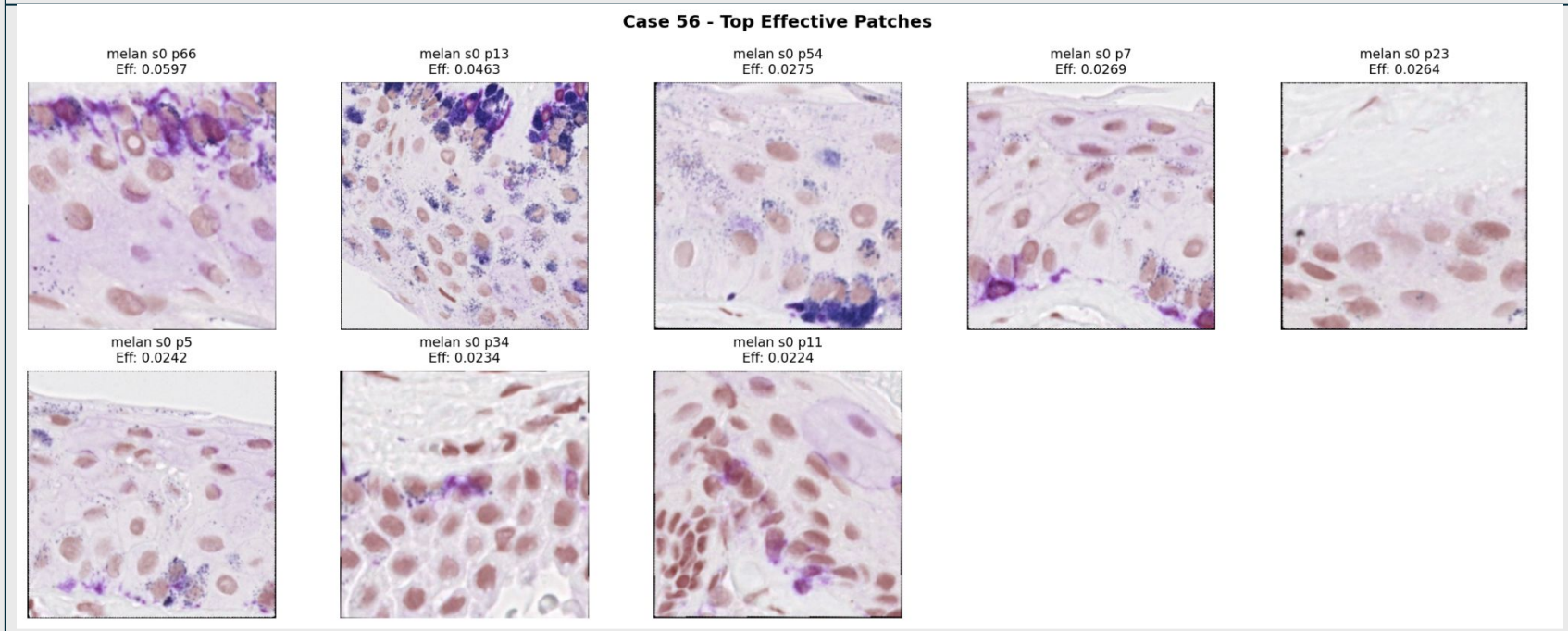
➔ More benigns misclassified, with **75-25** split among misclassifications

| Split | Misclassified Cases | Notes |
|---|---|---|
| 1 | **Benign**: 56 | |
| 2 | **Benign:** 109, 110, 86, 26<br>**High-Grade:** 32 | 109, 110 are “psuedocases”<br>Nearly all completely wrong |
| 3 | **Benign:** 93, 46<br>**High-Grade:** 51, 15 | Benigns completely wrong |
| 4 | None :) | |
| 5 | **Benign:** 96<br>**High-Grade:** 69 | |

**TAKEAWAYS:**

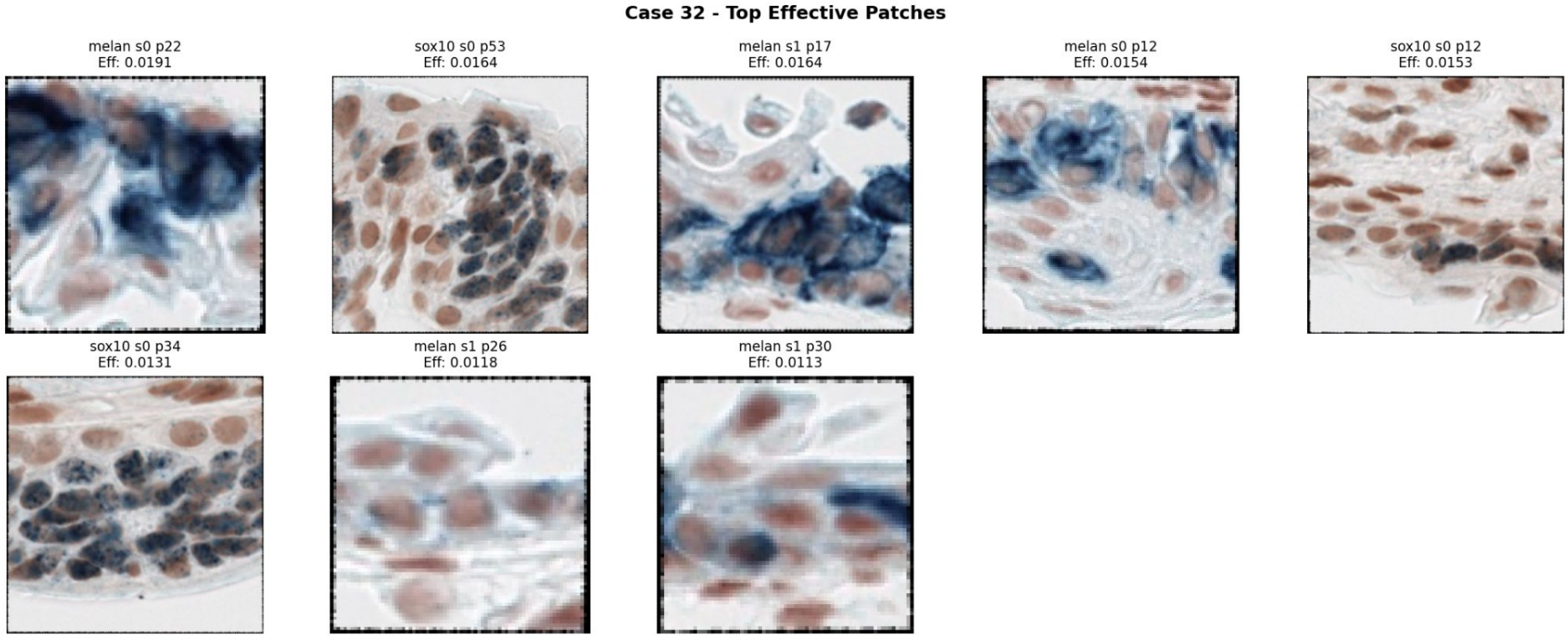
- ➔ Lack of stain/slice diversity — cases concentrated on a single stain or slice type, reducing generalisability
- ➔ Poor image quality in subset of cases likely contribute to errors
- ➔ Split 4 achieved zero misclassifications, suggesting its composition avoids these confounds

# Split 1: Patch Attention Analysis

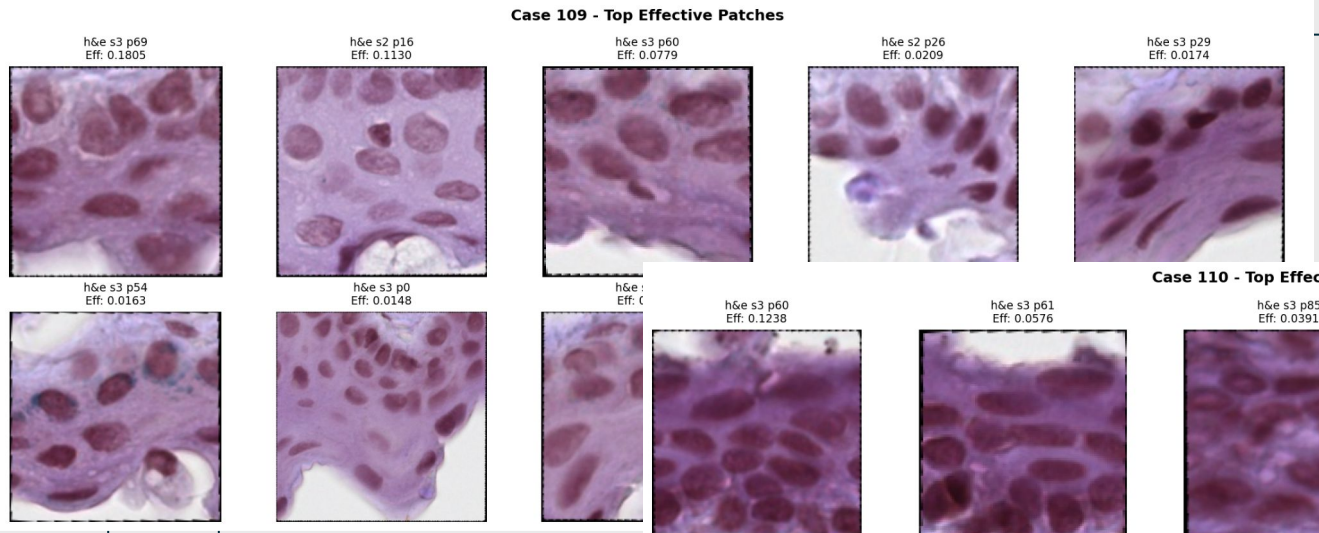


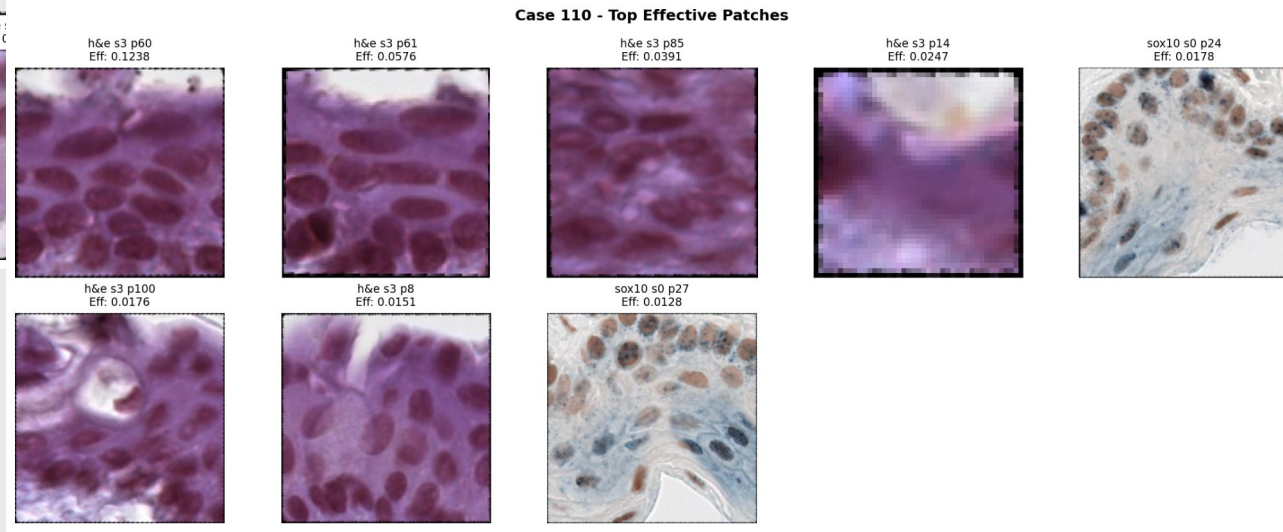

/projects/e32998/MIL_training/final_runs/run_20260314_172504/attention_analysis/case_effective_patches

# Split 2: Patch Attention Analysis



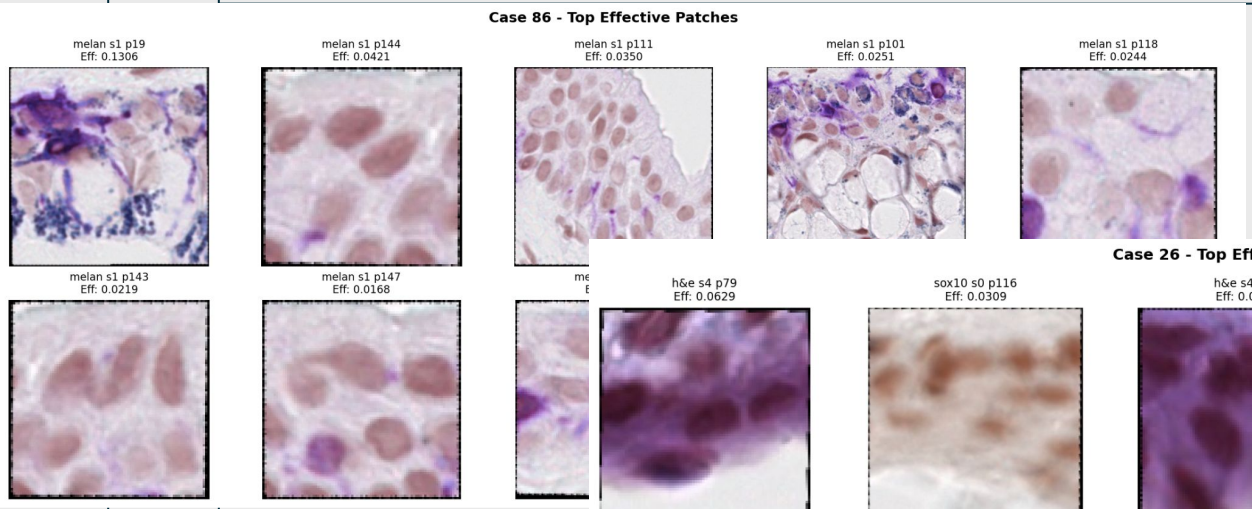
# Split 2: Patch Attention Analysis

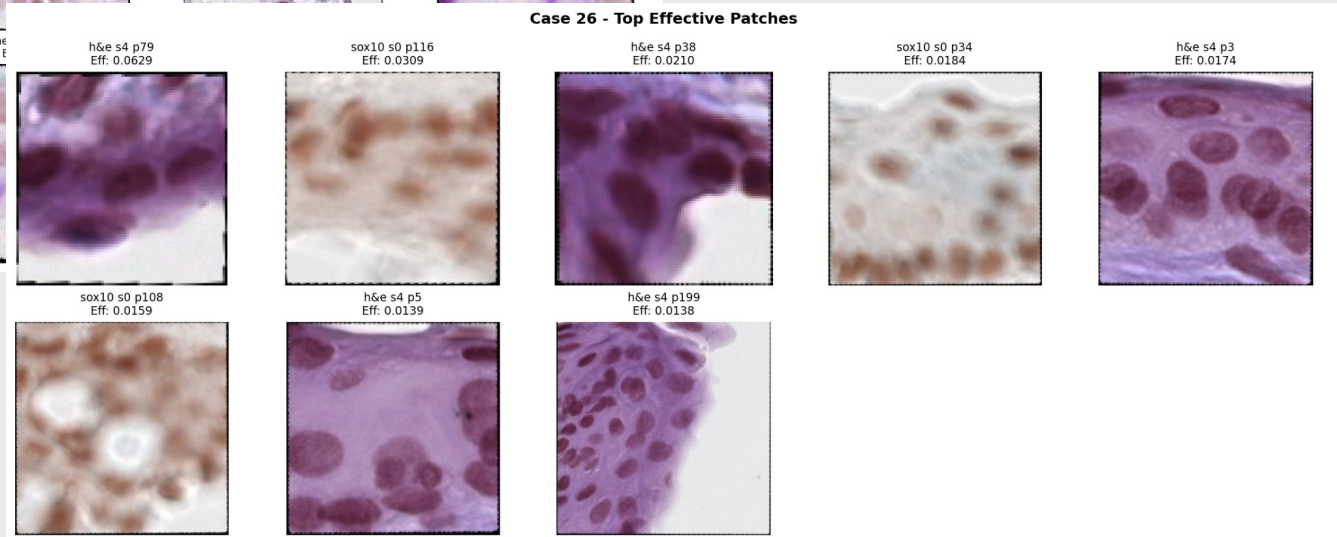



/projects/e32998/MIL_training/final_runs/run_20260314_174607/attention_analysis/case_effective_patches

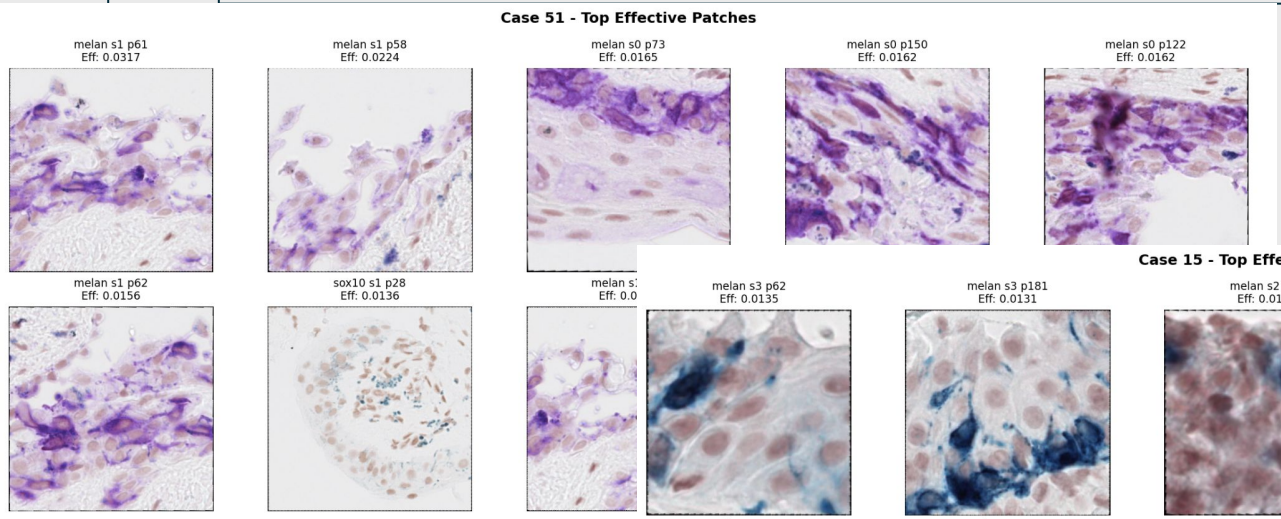
# Split 2: Patch Attention Analysis

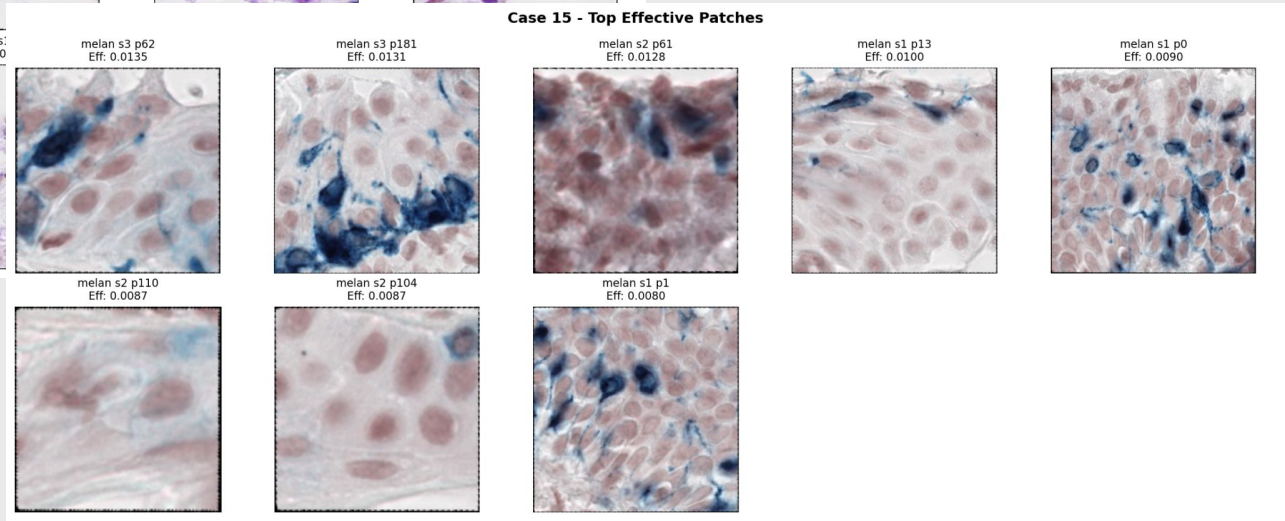



/projects/e32998/MIL_training/final_runs/run_20260314_174607/attention_analysis/case_effective_patches

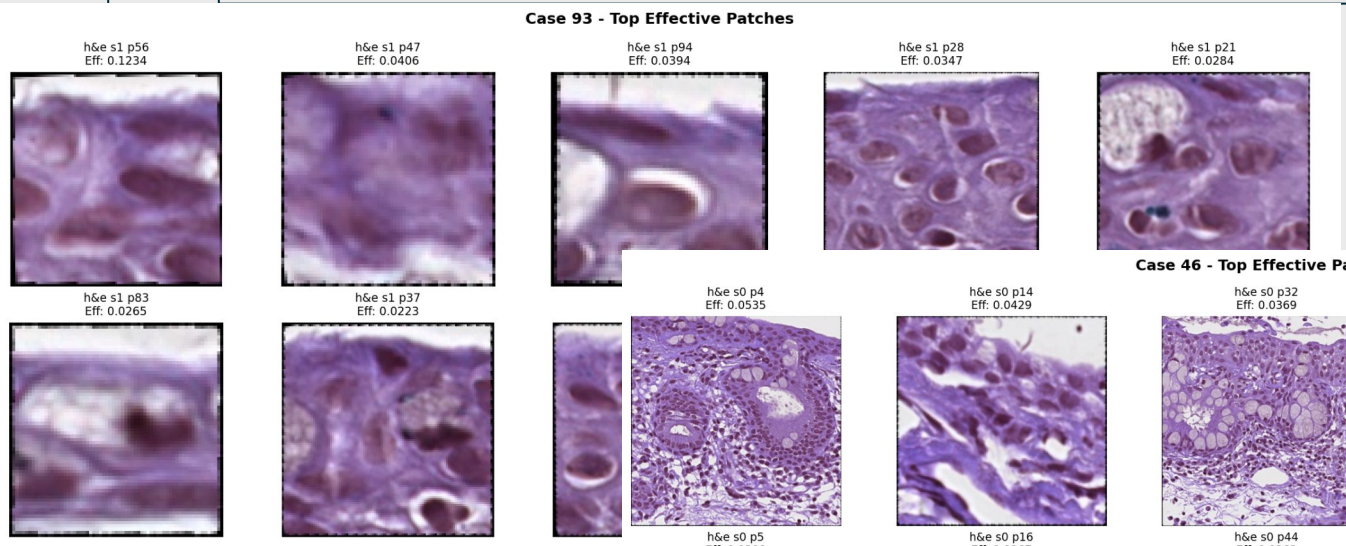
# Split 3: Patch Attention Analysis

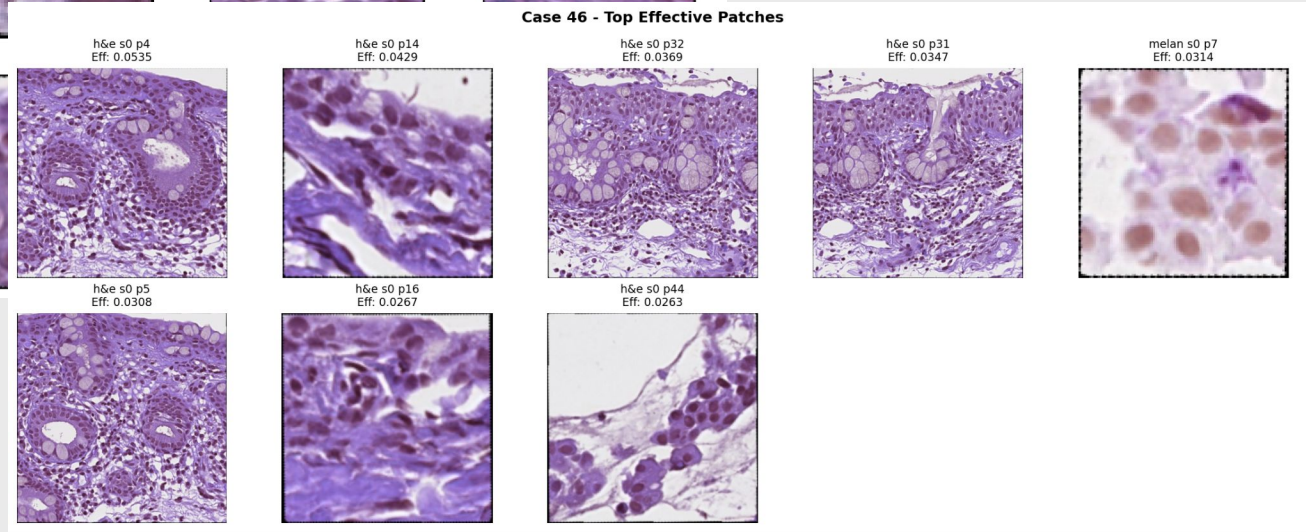



/projects/e32998/MIL_training/final_runs/run_20260314_180726/attention_analysis/case_effective_patches

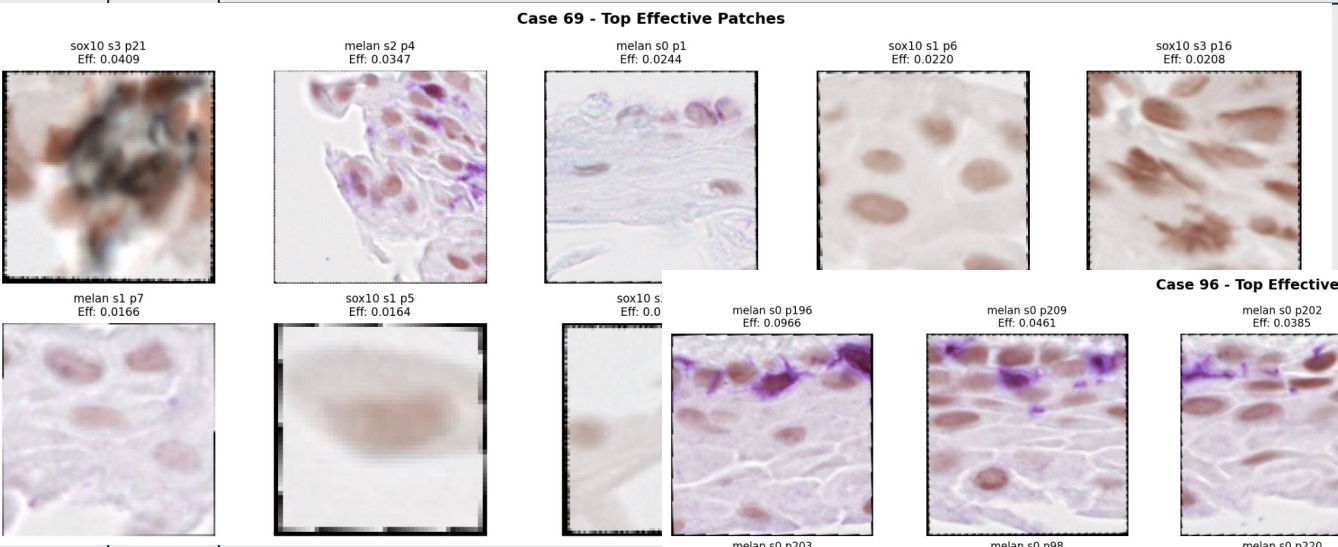
# Split 3: Patch Attention Analysis

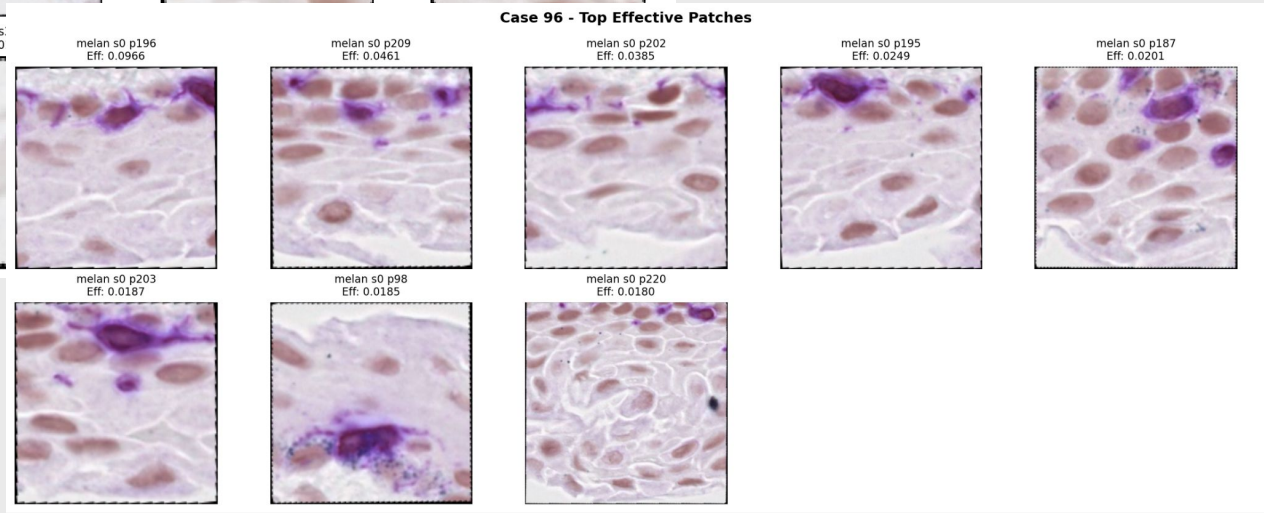



/projects/e32998/MIL_training/final_runs/run_20260314_180726/attention_analysis/case_effective_patches

# Split 5: Patch Attention Analysis




/projects/e32998/MIL_training/final_runs/run_20260314_184428/attention_analysis/case_effective_patches

# Hyperparameter Tuning

- **Takeaways:**
  - Learning during Optuna study plateaus after 5 trials
  - 5 fold cross validation very computationally expensive, the most beneficial next steps would be to aim to reduce runtime
  - Tuning appeared to make our model less generalizable, hurting accuracy across splits more than helping
  - Learning during training consistently plateaus around epoch 8

- **Recommendations for future changes:**
  - Parallelization of execution on folds in optuna script
  - Increase n_jobs for parallelization

# Similarity of Models

- **Goal:** assess if the five models trained on five different train-validation-test splits are similar (indicating stable learning and reproducible features), or differ vastly (indicating instability due to limited data and possible overfitting)
- **Proposed Solution to Quantify Similarity: Kullback-Leibler (KL) Divergence**
- **Steps**
  - **Identify Common/Reference Dataset for Comparison, *P***: The **Full Dataset** should be utilized as the reference dataset. This ensures that all available variations of bags, patches, and stain variations are seen. Using the entire dataset limits bias that can occur if only a subset of the dataset or a single split of the data is utilized (prevents misleading interpretations of similarity and dissimilarity). Additionally, due to limited ability to obtain new unseen data, best to use entire dataset.
  - **Identify Probability Vectors for Reference Data:** For the reference dataset, ***P***, need to compute the probabilities for high-grade and low-grade class seen in the entire dataset (ex: total number high cases / total cases). Resulting vector serves as $\boldsymbol{P_j}$ the full dataset distribution of high/low grade cases
  - **Probability Vector for Comparison Data:** For each model *i*, need to have predicted probability for all bags, *k*, belonging to a high grade case, this gives vector ***q***. (ex: $q_k^{(i)} = [q_k(low), q_k(high)]$ *k = bag index*
    - **Average Predicted Probabilities Across all Bags:** Need to average probabilities of all bags to be able to compare to ground truth values to obtain $Q_i = [Q_i(low), Q_i(high)]$
      *N= total number of bags/case*
      *Needed for both low and high grade*

$$Q_{i,low} = \frac{1}{N}\sum_{k=1}^{N} q_{k,low}$$

# Similarity of Models

- **Compute KL Divergence for relative train/test data from splits**
  - **Ex: Compute Divergence Value (difference between probability distributions) from full data set, *P*, to ith split training data set, *Q*. *Note: Order matters on which dataset probability is being utilized as reference. KL Divergence is *not* symmetrical***

$$D_{KL}(P||Q_{i,\text{train}}) = \sum_j P_j \log \frac{P_j}{Q_{i,j,\text{train}}}$$

  - Divergence value can be repeated for all *i* splits for train and test probability values to see how different probabilities are from the full dataset. Ex: Compute Divergence Value from full data set, *P*, to 3rd split on training/test data.
- **Interpreting Divergence Values:** The resulting value provides the number of "extra bits" required to correctly represent the data. Low divergence values indicate the model is stable, and is able to match to the reference dataset. High divergence values indicate the model's predictions deviate significantly from the reference dataset.
  - **Ideal Divergence Value is 0, meaning that both distributions at hand for comparison are identical.**
  - The closer divergence value is to 0, the the closer the model is able to match the reference dataset distribution. Goal is to have lower divergence values.

# Similarity of Models

### Why KL Divergence?

| | Measurement Purpose | Limitations |
|---|---|---|
| **Pearson Correlation** | Measures linear relationship between two variables/vectors | Measure relationship for variables in same direction, not if probability values are similar; does not penalize differences in scale |
| **Cosine Similarity** | Measures the similarity between two non-zero vectors by calculating cosine of the angle between them | Does not take into account magnitude difference; vectors can produce same similarity score but differ in scale; focuses on direction |
| **Jensen Shannon (JS) Divergence** | Measures similarity between two probability distributions in bounded and symmetric way | Reduced sensitivity to shifts in probability mass as a result of divergence difference being smoothed out before divergence score is calculated; bounded interval can cause large differences to be reduced or small |

# Similarity of Models

**Are predicted probabilities enough?**

- **No -** the final classification output is dependent on the **previous stage** and information within the MIL. Different values in feature embeddings, attention scores, and weights can produce the **same output** in the subsequent hierarchical layer, and vice versa. It is important to evaluate internal model quantities to ensure stable feature learning

**Evaluating Attention Score Distribution**

- **Why?** - Attention score weights determine how much each patch contributes to the slice, and which slice(s) contribute to the final bag/case classification. Evaluating attention scores reveal which image regions are influencing final classification. Different models may be producing the same classification, but attend to different patches.
- **JS Divergence** - JS works by being bounded  from 0 to 1 (using base 2 log algorithm). There is no 'true' distribution that enables one distribution of attention score's to be reference set, JS allows both distributions to be compared symmetrically. Since attention scores are normalized to values of (0,1), creates easier interpretation resulting divergence value (0 vs 0.1 vs 0.5).

# Similarity of Models

**Latent (Bag Embeddings) Representations**

- **Why? -**Embeddings represents how the model understands and immediately learns features. Different learned features can lead to similarly classification. Comparing Embeddings allows for direct comparison of learned features / structures.  Ensures that modes that produce the saem classification produce similar embeddings that indicate similar learned features.
- **Cosine Similarity -** Cosine focus on direction between the two vectors being compared, rather than magnitude. Embeddings can have different values in magnitude, but still point to the same direction, indicating same learned feature based on direction. Values closer to 1 indicate nearly identical vectors, values closer to 0 indicate less similarity (learned embeddings are point in different directions).

# Small Data Memorization

➔ **Overparameterization** – when the neural network has more parameters than necessary to fit the training data

$$L_{\text{train}}(\theta) = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(p_i) + (1-y_i)\log(1-p_i)]$$

$$p_i = \sigma(f_\theta(x_i)) \ \ [\text{activated (post-sigmoid) output}]$$
$$f_\theta(x_i): \ \text{raw logit}$$
$$\sigma(z) = \frac{1}{1+e^{-z}} \ \ [\text{sigmoid function}]$$

- Network can assign separate configurations of parameters for each training example, e.g. choose parameters such that

$$f_\theta(x_i) \gg 0 \text{ if } y_i = 1 \ \ (\text{i.e. high-grade})$$
$$f_\theta(x_i) \ll 0 \text{ if } y_i = 0 \ \ (\text{i.e. benign})$$

$$p_i \to 1 \ \ (\text{for y_i} = 1)$$
$$p_i \to 0 \ \ (\text{for y_i} = 0)$$

# Small Data Memorization

## ➔ Returning to BCE loss

$$p_i \to 1 \quad (\text{for y_i} = 1)$$
$$p_i \to 0 \quad (\text{for y_i} = 0)$$

$$L_{\text{train}}(\theta) = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(p_i) + (1 - y_i)\log(1 - p_i)]$$

- Substituting these limits in, we see the following:
  - For high-grade:

$$-\log(p_i) \to 0$$

  - For low-grade:

$$-\log(1 - p_i) \to 0$$

$$L_{\text{train}}(\theta) \to 0$$

Basically: model drives training loss arbitrarily close to zero by making incredibly confident predictions, i.e. memorizing.

# Small Data Memorization

- **Why poor generalization?**
  - Previous slides deal with **empirical risk** (i.e. training loss)
  - Now, we want to look at **expected risk** (i.e. testing loss)
    - If the model is just memorizing the training data, and not learning

- **Relation to observed loss curves**
  - Training loss rapidly decreases (memorization is quick!)
  - Approaches zero after a few epochs
    - Model is finding parameters such that $\boxed{f_\theta(x_i)}$ produces extremely confident predictions for all training cases (i.e. minimal BCE loss)
  - Validation loss is high/unstable

# Small Data Memorization

- **Case-level supervision**
  - Number of labeled cases ($n$) = number of individually supervised samples
    - Even as $m \to \infty$, where $m$ is number of patches in a cases, the number of labeled targets remains $n$
  - Model is constrained by $n$ cases, not $m \times n$ patches

- **Why MIL model can still memorize**
  - Can still learn hyperparameters such that:

$$f_\theta(x_i) \gg 0 \text{ if } C_i = 1 \ \ \text{(i.e. high-grade)}$$
$$f_\theta(x_i) \ll 0 \text{ if } C_i = 0 \ \ \text{(i.e. benign)}$$

  - Attention mechanism still in place → can assign large weights to small subsets of patches *within each case*
    - Model memorizes each case individually using case-specific patch features

# Small Data Memorization

- **Relation to observed loss curves**
  - Model memorizing each case in training data:

    $$L_{\text{train}}(\theta) \to 0$$

  - Not generalizable, so

    $$L_{\text{val}} \text{ remains high/unstable}$$

- **Assumptions being made**
  - Case-level supervision *only*
  - Patches within case are correlated, not independent.
  - Attention mechanism works as intended within each case.
    - i.e. can concentrate weight on subset of important patches.

# Takeaways

- Hyperparameter Tuning exhibited **limited benefit at high computational expense**
  - Plateaus after 5 trials, cross-validation too expensive
  - Consider **parallelizing folds** or **avoiding tuning** all together
- Patch attention analysis demonstrates trends among misclassifications
  - **Benign most misclassified**, potentially due to **poor image quality** & **lack of diversity** in stain/slice type
- **Different methodologies required** for different comparisons between 5 splits
  - **KL divergence** to compare **similarity**
  - **JS divergence** to compare **attention scores**
  - **Cosine** to compare **low-level embeddings**
- **Overparameterization** contributes to small-data memorization, **hurts generalizability**

# Project Reflection

**Technical Learnings:**

- Improve upon existing models (attention scoring, loss functions, hyperparameter tuning)
- GitHub & version control
- QUEST, running things on virtual server
- Pulling insights from research papers
- PyTorch & other ML libraries

**Non-technical learnings:**

- Communication is key (within members & to stakeholders)
- Setting deadlines and workflows (work influences future steps and other team members)

**Changes for Next Time:**

- Longer ramp-up period to understand prior work
- Utilize separate repositories from the beginning
- More methodological and isolated changes to clearly see what helps vs. hurts

**Takeaways:**

- Establish rules and expectations for organization and structure in beginning
- Always start early – cannot predict how long model will take to train or what errors may arise
- Use math to inform decision making, don’t just throw things at the wall and see what sticks

# Thank you!